

Mutual Management

Project Documentation

Case study: 132 - Grow Mutual Fund Investment

Release: 1.1.0

Documentation date: 5 October 2026

Platforms: Android 7.0 / API 24 or newer, and web

Maintainer: [kadamsahil2511](#)

Repository: github.com/kadamsahil2511/mutual-management

Live application: mutual-management.vercel.app

Abstract

A beginner-friendly app for exploring Indian mutual funds, setting goals and practising investment decisions. Built for **case study 132, "Grow Mutual Fund Investment"**, it brings fund research, planning, portfolio analysis and learning into one Flutter application for Android and the web.

Simulation scope: Accounts, cloud records and published fund data are real. Every contribution and SIP installment is an explicitly confirmed simulation. No money moves, and the app does not execute investment orders.

Contents

1. Project purpose and audience
2. Start using the app
3. Screens and capabilities
4. A complete demonstration
5. How the simulation works
6. Data, sources and freshness
7. Design and accessibility
8. Key terms
9. Technology and architecture

10. Run and configure locally
 11. Testing and release evidence
 12. Build and release
 13. Limitations and troubleshooting
 14. Academic mapping and project history
 15. Documentation index
-

1. Project purpose and audience

Learning about mutual funds often means moving between fund pages, calculators, educational resources and spreadsheets. Mutual Management brings a small, traceable selection of these tools together so a beginner can understand the process before making financial decisions elsewhere.

The project is intended for students, first-time learners and classroom demonstrations. Its objectives are to make fund information understandable, support goal-based practice, explain diversification and risk, and demonstrate a complete cross-platform application with persistent user records.

2. Start using the app

- **Web:** Open the [live application](#) in a browser.
- **Android:** Install the [signed 1.1.0 APK](#) on Android 7.0 / API 24 or newer. Android may require permission for the browser or file manager to install it. This is an APK distribution, not a Play Store listing.
- **Guests:** Explore funds, compare schemes, complete the risk questionnaire, read articles and use calculators.
- **Account holders:** Save goals, create SIP schedules, record simulations and see the same records on another signed-in device. New accounts start empty.
- **Updating:** Version 1.1.0 uses the same Android package and signing certificate as 1.0.0. Install it over the earlier release to retain the app's local state.

3. Screens and capabilities

On phones, the five primary tabs are **Home, Funds, Portfolio, Plans and More**. More contains account and learning tools. Wider screens use the desktop navigation.

3.1 Home / Overview

See simulated portfolio value, gain/loss, goal progress, the next planned SIP and quick actions.

3.2 Funds

Search six schemes, filter three categories and select two or three funds for comparison.

3.3 Fund details

Explore Overview, Performance and Holdings; inspect dated NAV, returns, benchmark figures, AUM, expense basis, managers and separate SIP/lump-sum minimums.

3.4 Risk comfort

Answer five questions about time horizon, emergency savings, reaction to loss, experience and flexibility. The result is an educational reflection.

3.5 Portfolio

Review derived holdings, contributed amounts, valuation, gain/loss, sector allocation and disclosed-security overlap.

3.6 Plans

Create/edit goals, choose monthly or quarterly SIP schedules, confirm due installments and cancel future recording.

3.7 Activity and receipts

Review recorded simulations and open their amount, units, NAV/date and effective-date details.

3.8 Account and recovery

Register, sign in, view account information, request a password reset and sign out.

3.9 Learn

Read six original articles and open official educational videos and sources.

3.10 Calculators

Explore SIP, lump-sum and goal estimates with editable assumptions.

3.11 Fixed deposits

Review dated published bank rates and tenures, open bank sources and estimate maturity.

3.12 More / About

Find tools, account access, contribution activity and an explanation of the simulation.

Goal creation/editing, SIP creation and contribution recording have dedicated mobile screens. Forms resize for the keyboard; dragging dismisses it. Details and forms provide Back navigation, and web routes support direct links.

4. A complete demonstration

1. Open **Funds**, search or swipe category filters, select two funds and compare them. Open a fund's details to inspect its NAV date and separately dated reference information.
2. Open **More** → **Risk comfort** and complete the five questions.
3. Open the account icon and create an account with a name, email and password of at least eight characters. The portfolio begins empty.
4. In **Plans** → **Goals** → **Create goal**, enter "Education", a ₹1,20,000 target and a future date. Keep the assumed annual return at 0% to see a contribution-only plan.
5. In **Plans** → **SIPs** → **Create SIP plan**, choose a fund, ₹1,000, a monthly schedule and a first date that is already due. Optionally link the goal.
6. Select the due installment, review the historical NAV/date and calculated units, then choose **Confirm simulation**. Nothing is recorded automatically.
7. Open **Portfolio**, inspect the holding and add a second fund through **Add simulation** to demonstrate overlap. Open **Activity** and a receipt.

8. Reload or sign in on a second device to demonstrate persistence and synchronization. Try a calculator, then sign out on shared devices.

Choose a recent due date with available NAV history; recording rejects an unusable price gap. A demonstration amount still requires review and confirmation. See the [detailed walkthrough](#).

5. How the simulation works

- **Contribution valuation:** Units equal the rupee amount divided by the published NAV used for that record.
- **One-off contributions:** Use the latest available published NAV, with date/freshness checks.
- **SIP installments:** Use NAV on or before the scheduled date, with a seven-day data-gap limit. Recording requires explicit confirmation.
- **Monthly and quarterly dates:** Keep the original day and clamp only the shorter month: January 31 → February 28/29 → March 31.
- **Duplicate protection:** A deterministic SIP/date record ID and Firestore transaction prevent the same installment being saved twice.
- **Portfolio totals:** Derive from immutable contribution records; there is no separately maintained balance.
- **Goal progress:** Tracks contributed amounts. Planning estimates use an editable assumed return, default 0%, and end-of-month contributions.
- **Returns:** 1Y is an absolute return; 3Y/5Y are annualized. Insufficient history stays unavailable.
- **Portfolio X-ray:** Sector weights use portfolio values; overlap sums the smaller disclosed weight for each matching security identifier.

A tested example: ₹1,000 at NAV ₹25 gives 40 units; ₹500 at NAV ₹20 adds 25 units. At NAV ₹24, the 65 units are worth ₹1,560, a ₹60 gain. This is a test example, not seeded account data.

6. Data, sources and freshness

The catalogue contains six **Direct Growth** schemes:

Category	HDFC scheme code	SBI scheme code
Flexi Cap	118955	119718
Corporate Bond	118987	146215
Aggressive Hybrid	119062	119609

- **NAV and history:** MFAPI responses are fetched by scheme code. The app displays the actual NAV valuation date separately from the retrieval/cache date. Successful public responses are cached for six hours, with refresh, a ten-second request timeout, one retry and labelled cached fallback.
- **Scheme references:** Expense basis, AUM, managers, benchmark comparisons and minimums come from dated AMC disclosures. The main factsheet snapshot is 31 August 2026; separately checked minimums retain their own source date.
- **Holdings:** Available top-holding disclosures are partial. Coverage and unreported assets remain visible; debt issuer aggregates are not presented as individual identified securities.
- **Expense comparison:** The HDFC and SBI disclosures use different expense bases. "Selected-category average · 2 funds" is shown as unavailable where a comparable average cannot be calculated.
- **Bank deposits:** Rates are bundled dated snapshots from HDFC Bank and SBI, not a live bank feed. The maturity calculator illustrates quarterly compounding before tax; bank terms determine actual proceeds.
- **Education:** Six original summaries cover mutual funds, NAV, risk, expenses, direct/regular plans and the Riskometer, with official AMFI/SEBI sources and videos.
- **Missing data:** Failed requests and unavailable fields remain explicit. The app does not generate replacement market figures.

The [data-source guide](#) records official source links, disclosure dates, expense definitions, minimum-investment provenance and coverage limitations.

7. Design and accessibility

The visual identity follows [DESIGN.md](#): blue `#0052ff`, white/gray surfaces, dark overview cards, Inter text, JetBrains Mono financial figures, 24px card corners, 12px inputs and pill-shaped actions.

Version 1.1 introduced a compact mobile dashboard, visible bottom tabs, horizontally scrollable fund filters, focused detail sections, separate forms, receipts and a More hub. Desktop retains the wider editorial layout. Content is capped at 1200px, with compact phone padding and adaptive card layouts.

Verification covers narrow screens, large desktop widths, 200% text scaling, 48px interaction targets and Android portrait/landscape. Financial rows stack when needed. The [mobile UX audit](#) explains the findings and fixes.

Figure 1. Mobile Home



Good afternoon,

Your money, in view.

SIMULATED PORTFOLIO

₹0.00

No contributions recorded

Practise with a contribution. No money is moved.

+ Add simulation

🔖 New goal

🔍 Find funds

No goals yet. Set a target for something that matters.

Create

No upcoming SIPs. Create a schedule when you are ready.

Plan



Home



Funds



Portfolio



Plans



More

Mobile Home screenshot

The compact dashboard shows an empty simulated portfolio, quick actions and persistent bottom navigation.

Figure 2. Fund Catalogue

Discover



Explore funds

Published data, clearly dated.

Search funds

✓ All funds

Flexi Cap

Corporate

6 schemes · Published NAV, with valuation dates

H

HDFC Flexi Cap Fund

Flexi Cap · HDFC Mutual Fund

Latest NAV

1-year return

₹2,150.0820

-2.77%

1 Oct 2026

Expense ratio · 0.67% · 31 Aug 2026

NAV retrieved · 5 Oct 2026

Explore fund

Compare

S

SBI Flexicap Fund

Flexi Cap · SBI Mutual Fund

Latest NAV

1-year return

₹117.1988

-3.93%



Home



Funds



Portfolio



Plans



More

Fund catalogue screenshot

Search and horizontal category filters lead to dated fund information and comparison actions.

Figure 3. Goal Creation



New goal

Create a goal

Set a target and a date that work for you.

Create a goal

Goal name

0/80

Target amount (₹)

Target date (YYYY-MM-DD)

2029-10-05

Assumed annual return (%)

0

Default 0%. This editable assumption is for planning; returns are not guaranteed.

Contributions are made at month end.

Save goal

Goal form screenshot

A dedicated form collects the target, date and editable return assumption. Saving requires an account.

[More hub](#) · [Fund details](#) · [Desktop overview](#) · [Android keyboard](#)

8. Key terms

The main financial terms used in the app are:

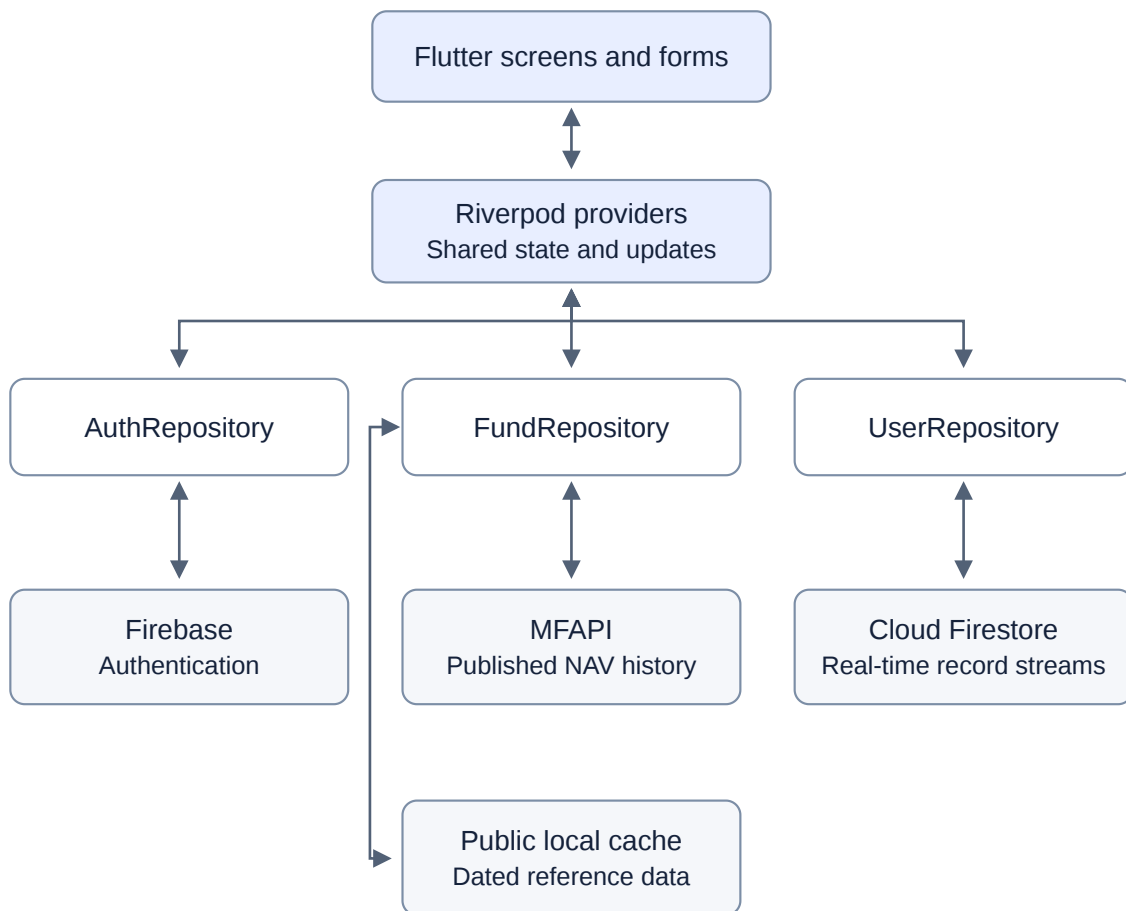
Term	Meaning in this project
NAV	Published value of one fund unit; used to calculate simulated units and valuation
SIP	A recurring contribution schedule; each due installment here needs manual confirmation
AMC	The asset management company managing a fund, such as HDFC Mutual Fund or SBI Mutual Fund
AUM	Assets under management reported for a scheme at its disclosure date
Benchmark	A reference index used for a separately dated performance comparison
CAGR	An annualized rate describing growth over a multi-year period
Overlap	Shared identified securities between two funds, limited to disclosed holdings

9. Technology and architecture

- **App and language:** Flutter 3.47.1 / Dart 3.13.1; shared web and Android application code
- **UI:** Material 3, ordinary widgets, bundled fonts and responsive layouts
- **Shared state:** Riverpod providers; local `setState` for temporary form and selection state
- **Navigation:** `go_router` for destinations, deep links and Back behavior
- **Authentication:** Firebase Authentication email/password accounts and session changes
- **Cloud records:** Cloud Firestore documents with real-time streams and owner-only rules

- **Public data:** `http` requests to MFAPI plus bundled dated reference snapshots
- **Local storage:** `shared_preferences` for public NAV cache
- **Utilities:** `intl` for currency/dates; `url_launcher` for external sources and videos
- **Delivery:** Vercel static web hosting; a signed Android APK; Git/GitHub releases
- **Verification:** Flutter tests/integration tests, Firestore Emulator, GitHub Actions; Node 22 and Java 21 for emulator testing

Figure 4. Application Architecture



Application architecture

Screens use Riverpod to exchange data through three repositories. The repositories connect to authentication, public fund data and the cloud records; record streams update the shared state.

The three concrete repositories keep network and database work separate from widgets. Saving a goal validates the form, checks sign-in, writes through `UserRepository`, and receives the updated record through a Firestore stream. Riverpod then updates Home and Plans. The app connects directly to Firebase and the public API; this release has no custom application server.

9.1 Code and data map

- `lib/features/`: auth, overview, funds, plans, portfolio, learn, deposits and More.
- `lib/core/`: providers, theme, shared widgets, input validation and calculations.
- `lib/app.dart`: routes and responsive navigation; `lib/main.dart`: startup.
- `assets/data/`: learning content and dated FD rates; `assets/fonts/`: bundled fonts and licenses.
- `test/`, `integration_test/`, `firebase-tests/`: calculation, UI, device and security checks.
- `firestore.rules`, `firebase.json`, `scripts/package-web.sh`: backend rules/emulators and web packaging.

Personal collections are scoped to `mutualManagementUsers/{uid}`:

- **Goal:** Name, target amount in integer paise, target date and assumed return
- **SIP:** Scheme code, amount in paise, start date, monthly/quarterly interval, optional goal and active/cancelled status
- **Contribution:** Amount, units, NAV, NAV date, effective date, scheme code and optional SIP/goal association

Authentication stores account identity and display name. Calendar dates use UTC midnight in stored models. Private Firestore disk persistence is disabled; SharedPreferences is used for public prices. Firebase Auth maintains the sign-in session until sign-out. Rules validate ownership, record shape and positive amounts, and contributions are immutable through client access.

10. Run and configure locally

Prerequisites: Flutter 3.47.1 / Dart 3.13.1, a browser, and an Android SDK/device for Android work. Node 22 and Java 21 are used for Firestore emulator checks.

```
git clone https://github.com/kadamsahil2511/mutual-management.git
cd mutual-management
flutter pub get
flutter run -d chrome
# For Android, choose an ID from flutter devices:
flutter run -d DEVICE_ID
```

The checked-in Firebase client configuration connects to `device-streaming-f3ea5c85`, displayed as Mutual Management, with Firestore in `asia-south1`. Client identifiers are not administrator credentials. Service-account keys, account passwords and release-signing secrets are excluded from the repository.

For an independent deployment, register Firebase web and Android apps, use Android package `com.kadamsahil.mutual_management`, replace `lib/firebase_options.dart`, update `.firebaserc`, enable Email/Password Auth, create Firestore and deploy the included rules. Add the deployed web hostname to Auth's authorized domains.

For local backend development, start Auth and Firestore emulators before running the emulator-enabled app:

```
npm ci --prefix firebase-tests
firebase-tests/node_modules/.bin/firebase emulators:start \
  --only auth,firestore --project demo-mutual-management
# In another terminal:
flutter run -d chrome --dart-define=USE_EMULATORS=true
```

Android emulator builds use host `10.0.2.2`; web uses `127.0.0.1`. Starting the local services alone does not redirect an ordinary app run; pass `USE_EMULATORS=true`.

11. Testing and release evidence

Version 1.1 evidence records **45 passing Flutter unit/widget tests, one Android API 35 integration test**, and **six Firestore security tests**. Tests cover validation, NAV failures/cache/retry, numerical calculations, duplicate recording, schedule boundaries, ownership and immutable records.

Responsive checks cover 320, 390, 640, 768, 1024, 1050, 1100, 1280 and 1920px, including 200% text. The signed APK was installed and launched on API 35; portrait, landscape and keyboard behavior were inspected. API 24 is the supported build minimum, not a device-test claim.

```
dart format --output=none --set-exit-if-changed \  
  lib test integration_test  
flutter analyze  
flutter test  
npm ci --prefix firebase-tests  
firebase-tests/node_modules/.bin/firebase emulators:exec \  
  --only firestore --project demo-mutual-management \  
  'npm --prefix firebase-tests test'  
flutter test integration_test/app_test.dart -d emulator-5554
```

The Firestore test command uses a demo project. Android integration requires a running device with the selected ID. GitHub Actions runs formatting, analysis, Flutter tests, a web build and Firestore security tests on pushes and pull requests; Android device testing is run separately.

Production verification also covered registration, account isolation, two-client synchronization, goal/SIP creation, explicit contribution recording and persistence. Details and historical release distinctions are in [verification evidence](#).

12. Build and release

```
./scripts/package-web.sh  
vercel link --project mutual-management --scope ksahil-team  
vercel deploy --prebuilt  
# After checking the preview:  
vercel deploy --prebuilt --prod
```

The script packages Flutter's static web build using Vercel Build Output API v3, with a filesystem-first fallback to `index.html` for application routes.

```
MUTUAL_SIGNING_PROPERTIES=/absolute/private/path/signing.pr  
operties \  

```

```
flutter build apk --release
```

The external properties file contains `storeFile`, `storePassword`, `keyAlias` and `keyPassword`. Keep it and the keystore outside Git and back them up privately for future updates. Without this variable, local builds use development signing. The signed APK is produced at `build/app/outputs/flutter-apk/app-release.apk`.

Release 1.1.0 uses version code 2 and supports Android API 24+. Its [release page](#) includes the APK and checksum file.

13. Limitations and troubleshooting

13.1 A new portfolio is empty

This is expected. Create a plan or explicitly confirm a simulated contribution.

13.2 A price is cached or unavailable

Check its NAV date, restore connectivity and refresh/retry. Existing public cache may be shown with a warning.

13.3 An installment cannot be recorded

Check the scheduled date, available history and the seven-day price-gap limit. Already-recorded installments cannot be duplicated.

13.4 Goal estimate seems high

Check target/date and the assumed return. The default is 0%; no growth is assumed.

13.5 Allocation or overlap is incomplete

Only disclosed, identified holdings are included. Coverage and unreported assets are shown.

13.6 The expense average is unavailable

The selected disclosures use incompatible expense definitions; the app avoids a misleading average.

13.7 A form asks for sign-in

Saving personal records requires an account. After sign-in, the pending form remains available.

13.8 Sign-in or reset fails

Check email, password and connectivity. Use the reset screen; on custom deployments verify Firebase Auth configuration and authorized domains.

This release has six selected funds, manually curated reference/rate snapshots, partial holdings and educational risk guidance. There are no automatic debits, withdrawals, bank connections, real fund orders, live FD bookings or stock trading. The pricing-reference panel reproduces labelled case-study examples, not financial offers. Calculator outputs are assumptions, not promised returns.

Account deletion/export, additional funds, broader complete holdings coverage and automated reference refresh are **possible future work**, not delivered features.

14. Academic mapping and project history

The software implements case study 132 and the final assignment's app requirements: multiple screens, Riverpod, API/Firestore integration, validation, local storage, Material 3, feature folders, repository separation and automated checks. See the [syllabus mapping](#) and [source requirements](#).

Version 1.0 established the end-to-end application and verified cloud/data flows. Version 1.1 addressed the mobile UX audit with app navigation and focused task screens. Both releases and genuine incremental commits are preserved. Classroom participation, handwritten work, peer assessment and the instructor's live surprise-feature exercise are separate activities.

15. Documentation index

1. [Demo guide](#) - Installation, walkthrough and presentation points
2. [Mobile UX audit](#) - Mobile findings, implemented changes and current screenshots
3. [Design guide](#) - Brand, colors, typography, spacing and component direction
4. [Data sources](#) - Provenance, dated disclosures and data limitations
5. [Data verification](#) - Detailed factual checks recorded during implementation
6. [Verification evidence](#) - Release, backend, security, browser and device evidence

7. UI baseline / Android baseline - Version 1.0 checks, retained as historical evidence
8. Syllabus mapping / Source requirements - Academic scope and requirement coverage
9. Original plan / Implementation contracts - Historical implementation records; the current code and this handbook describe version 1.1

Release downloads: Android APK and checksum.

Automated checks: GitHub Actions.